

JavaScript Arrays (Part 1)

This section covers:

- What is an Array?
 - Creating Arrays
 - Accessing Array Elements
 - Updating Elements
 - Adding Elements
 - Removing Elements
 - Array Length
-

What is an Array?

| Definition

An **Array** is a special JavaScript object used to store **multiple values in a single variable**.

Each value inside an array is called an **element**.

Every element has an **index**, and indexing starts from **0**.

| Why Do We Use Arrays?

Instead of creating multiple variables,

```
let subject1 = "Math";  
let subject2 = "English";  
let subject3 = "Physics";
```

Use an array.

```
const subjects = ["Math", "English", "Physics"];
```

Arrays make code:

- Easier to read
- Easier to manage
- Easier to loop through
- Easier to modify

Array Structure

```
const fruits = [  
  
  "Apple",  
  
  "Mango",  
  
  "Orange"  
  
];
```

Index

0

1

2

↓

Apple

Mango

Orange

Creating Arrays

There are several ways to create arrays.

| Method 1 (Most Common)

```
const fruits = [  
  
  "Apple",  
  
  "Mango",  
  
  "Orange"  
  
];  
  
console.log(fruits);
```

OUTPUT

```
["Apple", "Mango", "Orange"]
```

| Method 2 (Empty Array)

```
const numbers = [];  
  
console.log(numbers);
```

OUTPUT

```
[]
```

Method 3 (Mixed Data Types)

```
const data = [  
  "Rokon",  
  23,  
  true,  
  null  
];  
  
console.log(data);
```

OUTPUT

```
["Rokon",23,true,null]
```

Accessing Array Elements

Definition

Array elements are accessed using their **index number**.

Remember:

The first element starts at index **0**.

Syntax

```
array[index]
```

Example

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits[0]);  
  
console.log(fruits[1]);  
  
console.log(fruits[2]);
```

OUTPUT

Apple

Mango

Orange

Access Last Element

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits[fruits.length - 1]);
```

OUTPUT

Orange

| Access Non-Existing Index

```
const fruits = [  
  
  "Apple",  
  
  "Mango"  
  
];  
  
console.log(fruits[10]);
```

OUTPUT

```
undefined
```

Updating Elements

| Definition

You can change an existing element by assigning a new value to its index.

Syntax

```
array[index] = newValue;
```

Example

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
fruits[1] = "Banana";  
  
console.log(fruits);
```

OUTPUT

```
["Apple","Banana","Orange"]
```

Update Multiple Elements

```
const numbers = [  
  
  10,  
  
  20,  
  
  30  
  
];  
  
numbers[0] = 100;  
  
numbers[2] = 300;  
  
console.log(numbers);
```

OUTPUT

```
[100,20,300]
```

Adding Elements

There are different ways to add elements.

The most common method is **push()**.

(We'll learn `push()` in detail in Part 2.)

Example

```
const fruits = [  
  "Apple",  
  "Mango"  
];  
  
fruits.push("Orange");  
  
console.log(fruits);
```

OUTPUT

```
["Apple", "Mango", "Orange"]
```

Add Multiple Elements

```
const numbers = [  
  
  1,  
  
  2  
  
];  
  
numbers.push(3,4,5);  
  
console.log(numbers);
```

OUTPUT

```
[1,2,3,4,5]
```

Removing Elements

The most common method is **pop()**.

(We'll learn `pop()` in detail in Part 2.)

Example

```
const fruits = [  
  
  "Apple",  
  
  "Mango",  
  
  "Orange"  
];  
  
fruits.pop();  
  
console.log(fruits);
```

OUTPUT

```
["Apple", "Mango"]
```

Array Length

Definition

The `length` property returns the total number of elements in an array.

Syntax

```
array.length
```

Example

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.length);
```

OUTPUT

```
3
```

| Get Last Element Using length

```
const colors = [  
  "Red",  
  "Blue",  
  "Green",  
  "Black"  
];  
  
console.log(colors[colors.length - 1]);
```

OUTPUT

```
Black
```

Real Project Example 1

| Student Subjects

```
const subjects = [  
  "Math",  
  "English",  
  "Physics"  
];  
  
console.log(subjects[1]);
```

OUTPUT

```
English
```

Real Project Example 2

| Shopping Cart

```
const cart = [  
  "Laptop",  
  "Mouse"  
];  
  
cart.push("Keyboard");  
  
console.log(cart);
```

OUTPUT

```
["Laptop", "Mouse", "Keyboard"]
```

Real Project Example 3

| Remove Completed Task

```
const tasks = [  
  "Study",  
  "Exercise",  
  "Sleep"  
];  
  
tasks.pop();  
  
console.log(tasks);
```

OUTPUT

```
["Study","Exercise"]
```

Real Project Example 4

| Total Products

```
const products = [  
  "Phone",  
  "Laptop",  
  "Monitor",  
  "Keyboard"  
];  
  
console.log(products.length);
```

OUTPUT

```
4
```

Best Practices

✓ Use `const` when creating arrays.

```
const numbers = [];
```

- ✓ Use meaningful variable names.

```
students  
  
products  
  
orders  
  
employees
```

- ✓ Access the last element using:

```
array[array.length - 1]
```

- ✓ Keep similar data inside one array.

```
["HTML", "CSS", "JavaScript"]
```

Common Mistakes

| Wrong First Index

Wrong

```
fruits[1]
```

for the first element.

Correct

```
fruits[0]
```

| Accessing an Invalid Index

```
console.log(fruits[100]);
```

Output

```
undefined
```

| Forgetting Square Brackets

Wrong

```
const fruits = {};
```

Correct

```
const fruits = [];
```

| Confusing Array Length with Last Index

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.length);
```

Output

3

Last Index

2

Interview Questions

| What is an Array?

An array is a collection of multiple values stored in a single variable.

| What is the first index of an array?

0

| How do you access the first element?

```
array[0]
```

| How do you access the last element?

```
array[array.length - 1]
```

| How do you update an array element?

```
array[index] = value;
```

| Which method adds an element to the end?

```
push()
```

Which method removes the last element?

```
pop()
```

Which property returns the total number of elements?

```
length
```

Summary Table

Topic	Description	Example
Array	Collection of multiple values	<code>["A", "B"]</code>
Create	Create an array	<code>[]</code>
Access	Get an element	<code>array[0]</code>
Update	Change an element	<code>array[1] = "New"</code>
Add	Add to the end	<code>push()</code>
Remove	Remove from the end	<code>pop()</code>
Length	Total elements	<code>array.length</code>
Last Element	Access last item	<code>array[array.length - 1]</code>

Most Frequently Used



- Array Creation
- Accessing Elements
- Updating Elements
- `push()`
- `pop()`
- `length`



- Mixed Arrays
- Last Element Access

Final Notes

Arrays are one of the most important data structures in JavaScript. They are used extensively in:

- React Lists
- API Responses
- Shopping Carts
- User Data
- Product Lists
- Database Records

Mastering array creation, indexing, updating, and the `length` property is essential before learning advanced array methods and looping.

JavaScript Array Method - includes()

The `includes()` method is one of the most useful array methods in JavaScript. It is used to check whether an array contains a specific element.

Unlike `indexOf()`, `includes()` returns a **Boolean value** (`true` or `false`), making it simple and readable.

Table of Contents

- What is includes()?
 - Why Use includes()?
 - Syntax
 - Parameters
 - Return Value
 - How includes() Works
 - Basic Examples
 - Multiple Examples
 - Real Project Examples
 - Best Practices
 - Common Mistakes
 - Interview Questions
 - Summary Table
-

What is includes()?

Definition

The `includes()` method checks whether a specified element exists inside an array.

If the element exists, it returns

```
true
```

Otherwise, it returns

```
false
```

It **does not modify** the original array.

Why Use includes()?

We use `includes()` when we need to:

- Check if an item exists
 - Validate user input
 - Check permissions
 - Verify selected products
 - Prevent duplicate values
 - Search simple values in an array
-

Syntax

```
array.includes(value);

array.includes(value, startIndex);
```

Parameters

Parameter	Description
value	The value to search for
startIndex	Optional. Index to start searching from

Return Value

Returns

```
true
```

if the value exists.

Returns

```
false
```

if the value does not exist.

How includes() Works

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
includes("Mango")
```

↓

Result

```
true
```

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
includes("Banana")
```

↓

Result

```
false
```

Example 1 (Basic)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.includes("Mango"));
```

OUTPUT

```
true
```

EXPLANATION

"Mango" exists inside the array.

Example 2 (Value Not Found)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.includes("Banana"));
```

OUTPUT

```
false
```

Example 3 (Numbers)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
console.log(numbers.includes(30));
```

OUTPUT

```
true
```

Example 4 (Boolean)

```
const values = [  
  true,  
  false  
];  
  
console.log(values.includes(true));
```

OUTPUT

```
true
```

Example 5 (Using startIndex)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
console.log(numbers.includes(20,2));
```

OUTPUT

```
false
```

EXPLANATION

Searching starts from index **2**.

Index 2 contains

```
30
```

So

```
20
```

is not found.

Example 6 (Empty Array)

```
const data = [];  
  
console.log(data.includes(100));
```

OUTPUT

```
false
```

Example 7 (Case Sensitive)

```
const fruits = [  
  "Apple",  
  "Mango"  
];  
  
console.log(fruits.includes("apple"));
```

OUTPUT

```
false
```

EXPLANATION

JavaScript is case-sensitive.

```
Apple
```

is different from

```
apple
```

Example 8 (Duplicate Values)

```
const numbers = [  
  10,  
  20,  
  20,  
  30  
];  
  
console.log(numbers.includes(20));
```

OUTPUT

```
true
```

Real Project Example 1

| Shopping Cart

```
const cart = [  
  "Laptop",  
  "Mouse"  
];  
  
console.log(cart.includes("Laptop"));
```

OUTPUT

```
true
```

Real Project Example 2

| User Role

```
const roles = [  
  "Admin",  
  "Editor",  
  "User"  
];  
  
console.log(roles.includes("Admin"));
```

OUTPUT

```
true
```

Real Project Example 3

| Prevent Duplicate Student

```
const students = [  
  "Rahim",  
  "Karim"  
];  
  
const newStudent = "Rahim";  
  
if(!students.includes(newStudent)){  
  students.push(newStudent);  
}  
  
console.log(students);
```

OUTPUT

```
["Rahim","Karim"]
```

Real Project Example 4

| Allowed Countries

```
const countries = [  
  "Bangladesh",  
  "India",  
  "Japan"  
];  
  
console.log(countries.includes("Bangladesh"));
```

OUTPUT

```
true
```

Real Project Example 5

| Favorite Programming Languages

```
const languages = [  
  "JavaScript",  
  "Python",  
  "Java"  
];  
  
console.log(languages.includes("Python"));
```

OUTPUT

```
true
```

Best Practices

Use `includes()` when you only need to know whether a value exists.

Use meaningful variable names.

```
products
```

```
students
```

```
roles
```

```
languages
```

Use `includes()` instead of `indexOf()` for readability.

```
if(products.includes("Laptop")){  
  
}
```

Common Mistakes

| Mistake 1

Expecting `includes()` to return an index.

Wrong

```
const result = numbers.includes(20);  
  
console.log(result);
```

Output

```
true
```

It returns a Boolean, **not an index**.

| Mistake 2

Ignoring case sensitivity.

```
const fruits = ["Apple"];

console.log(fruits.includes("apple"));
```

Output

```
false
```

| Mistake 3

Using `includes()` to find an object.

```
const users = [  
  
  {  
  
    name: "John"  
  
  }  
  
];  
  
console.log(users.includes({  
  
  name: "John"  
  
}));
```

Output

```
false
```

Objects are compared by **reference**, not by value.

Mistake 4

Confusing `includes()` with `indexOf()`.

```
includes()
```

returns

```
true/false
```

while

```
indexOf()
```

returns

```
index / -1
```

Interview Questions

| What does `includes()` do?

Checks whether a value exists in an array.

| What does `includes()` return?

```
true
```

or

```
false
```

| Does `includes()` modify the original array?

No.

| Is `includes()` case-sensitive?

| Yes.

| Can `includes()` search from a specific index?

Yes.

```
array.includes(value, startIndex);
```

| Which method returns an index?

```
indexOf()
```

| Which method returns true or false?

```
includes()
```

Summary Table

Feature	Description
Method	<code>includes()</code>
Purpose	Check whether a value exists
Parameters	value, startIndex (optional)
Returns	<code>true</code> or <code>false</code>
Changes Original Array	No
Common Uses	Validation, Search, Permissions, Duplicate Check

Quick Comparison

Method	Returns	Modifies Array
<code>includes()</code>	<code>true</code> / <code>false</code>	No
<code>indexOf()</code>	Index / <code>-1</code>	No
<code>lastIndexOf()</code>	Last Index / <code>-1</code>	No

Final Notes

The `includes()` method is one of the easiest and most readable ways to check if an array contains a value. It is widely used in:

- Form validation
- Shopping cart applications
- Authentication & authorization
- Duplicate prevention
- Search functionality
- Permission checking

Mastering `includes()` will make your JavaScript code cleaner, more readable, and easier to maintain.

JavaScript Array Method - `indexOf()`

The `indexOf()` method is one of the most commonly used array methods in JavaScript. It is used to find the **index (position)** of a specific element in an array.

If the element is not found, it returns **-1**.

Table of Contents

- What is indexOf()?
 - Why Use indexOf()?
 - Syntax
 - Parameters
 - Return Value
 - How indexOf() Works
 - Basic Examples
 - Multiple Examples
 - Real Project Examples
 - Best Practices
 - Common Mistakes
 - Interview Questions
 - Summary Table
-

What is indexOf()?

Definition

The `indexOf()` method searches an array for a specified element and returns the **index of its first occurrence**.

If the element does not exist, it returns

-1

It **does not modify** the original array.

Why Use indexOf()?

We use `indexOf()` when we need to:

- Find an element's position
 - Check if a value exists
 - Remove an item using its index
 - Update an element
 - Search data inside an array
-

Syntax

```
array.indexOf(value);  
  
array.indexOf(value, startIndex);
```

Parameters

Parameter	Description
value	The value to search for
startIndex	Optional. The index to start searching from

Return Value

Returns

- Index of the first matching element
- `-1` if the value is not found

How indexOf() Works

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
indexOf("Mango")
```

↓

Result

```
1
```

Array

```
["Apple", "Mango", "Orange"]
```

↓

```
indexOf("Banana")
```

↓

Result

```
-1
```

Example 1 (Basic)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.indexOf("Mango"));
```

OUTPUT

1

EXPLANATION

"Mango" is located at index 1.

Example 2 (Value Not Found)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
console.log(fruits.indexOf("Banana"));
```

OUTPUT

-1

Example 3 (Numbers)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
console.log(numbers.indexOf(30));
```

OUTPUT

```
2
```

Example 4 (Duplicate Values)

```
const numbers = [  
  10,  
  20,  
  20,  
  30  
];  
  
console.log(numbers.indexOf(20));
```

OUTPUT

```
1
```

EXPLANATION

`indexOf()` always returns the **first matching index**.

Example 5 (Using startIndex)

```
const numbers = [  
  10,  
  20,  
  30,  
  20  
];  
  
console.log(numbers.indexOf(20,2));
```

OUTPUT

```
3
```

EXPLANATION

Searching starts from index **2**, so the second **20** is found.

Example 6 (Empty Array)

```
const data = [];  
  
console.log(data.indexOf(100));
```

OUTPUT

```
-1
```

Example 7 (Case Sensitive)

```
const fruits = [  
  "Apple",  
  "Mango"  
];  
  
console.log(fruits.indexOf("apple"));
```

OUTPUT

```
-1
```

EXPLANATION

JavaScript is case-sensitive.

Example 8 (Store Index)

```
const colors = [  
  "Red",  
  "Green",  
  "Blue"  
];  
  
const index = colors.indexOf("Blue");  
  
console.log(index);
```

OUTPUT

2

Real Project Example 1

| Remove a Product

```
const cart = [
  "Laptop",
  "Mouse",
  "Keyboard"
];

const index = cart.indexOf("Mouse");

cart.splice(index, 1);

console.log(cart);
```

OUTPUT

```
["Laptop", "Keyboard"]
```

Real Project Example 2

| Check Student

```
const students = [  
  "Rahim",  
  "Karim",  
  "Rokon"  
];  
  
console.log(students.indexOf("Karim"));
```

OUTPUT

```
1
```

Real Project Example 3

| Update Status

```
const status = [  
  "Pending",  
  "Approved",  
  "Rejected"  
];  
  
const index = status.indexOf("Approved");  
  
status[index] = "Completed";  
  
console.log(status);
```

OUTPUT

```
[  
  "Pending",  
  "Completed",  
  "Rejected"  
]
```

Real Project Example 4

| Search Category

```
const categories = [  
  "HTML",  
  "CSS",  
  "JavaScript"  
];  
  
console.log(categories.indexOf("CSS"));
```

OUTPUT

```
1
```

Real Project Example 5

| User Role

```
const roles = [  
  "User",  
  "Editor",  
  "Admin"  
];  
  
console.log(roles.indexOf("Admin"));
```

OUTPUT

```
2
```

Best Practices

✓ Use `indexOf()` when you need the position of an element.

✓ Always check for `-1`.

```
const index = array.indexOf(value);

if(index !== -1){

    console.log("Found");

}
```

✓ Use meaningful variable names.

```
productIndex

studentIndex

userIndex
```

Common Mistakes

| Mistake 1

Ignoring `-1`.

Wrong

```
const index = fruits.indexOf("Banana");  
  
fruits[index] = "Apple";
```

Always check

```
if(index !== -1)
```

Mistake 2

Expecting the last occurrence.

```
const arr = [10,20,20];  
  
console.log(arr.indexOf(20));
```

Output

```
1
```

Use `lastIndexOf()` for the last occurrence.

Mistake 3

Using `indexOf()` with objects.

```
const users = [  
  {  
    name: "John"  
  }  
];  
  
console.log(users.indexOf({  
  name: "John"  
}));
```

Output

```
-1
```

Objects are compared by reference.

Mistake 4

Ignoring case sensitivity.

```
["Apple"].indexOf("apple");
```

Output

```
-1
```

Interview Questions

| What does `indexOf()` return?

The index of the first matching element.

| What does it return if the element is not found?

```
-1
```

| Does `indexOf()` modify the original array?

No.

| Can it search from a specific index?

Yes.

```
array.indexOf(value, startIndex);
```

| Does `indexOf()` return the first or last occurrence?

The **first** occurrence.

| Which method returns the last occurrence?

```
lastIndexOf()
```

Which method returns `true` or `false` instead of an index?

```
includes()
```

Summary Table

Feature	Description
Method	<code>indexOf()</code>
Purpose	Find the first index of a value
Parameters	value, startIndex (optional)
Returns	Index or <code>-1</code>
Changes Original Array	No
Common Uses	Search, Update, Remove, Validation

Quick Comparison

Method	Returns	Modifies Array
<code>includes()</code>	<code>true</code> / <code>false</code>	No
<code>indexOf()</code>	First Index / <code>-1</code>	No
<code>lastIndexOf()</code>	Last Index / <code>-1</code>	No

Final Notes

The `indexOf()` method is essential when you need the **position** of an element rather than just checking if it exists. It is widely used in:

- Search functionality
- Updating array elements
- Removing items with `splice()`
- Data validation
- Form processing
- Menu and navigation systems

Mastering `indexOf()` is an important step before learning more advanced array methods such as `find()`, `findIndex()`, and `filter()`.